PAGE 1 - Project Introduction & How to Pitch

Master the initial 2 minutes of your technical placement interview with tailored elevator pitches.

Project Name	Supply Chain Product Tracking System
Problem Statement	Traditional logistics lack centralized, real-time visibility. Tracking shipments across production and distribution causes dispatch delays, misplaced goods, and untraceable supply chain status.
Main Objective	Develop an enterprise RESTful backend service to register products and record their strict milestone transitions (**Manufactured -> Shipped -> Delivered**) with zero data loss.
Real-World Application	Manufacturing hubs, e-commerce order fulfillment centers, and third-party logistics (3PL) warehouse networks.
Target Users	Plant warehouse managers, inventory officers, logistics dispatch coordinators.
Core Tech Stack	**Java 21, Spring Boot 3.3.5** (Spring Web, Spring Data JPA, Hibernate), **PostgreSQL**, Apache Maven, Jakarta Validation.

How to Explain This Project in an Interview (Choose Your Version)

1. The 30-Second Quick Pitch (When asked: 'Briefly introduce your project')

30-Second Elevator Pitch:

*"In my final semester, I developed a **Supply Chain Product Tracking System** using **Java 21, Spring Boot 3, and PostgreSQL**. The application provides RESTful APIs to track products across critical supply chain stages-from Manufacturing to Shipping and Delivery. I built the complete layered backend architecture consisting of Controllers, Services, and Spring Data JPA Repositories to ensure thread-safe, robust data persistence."*

2. The 1-Minute Pitch (Standard answer for 'Tell me about your project')

1-Minute Pitch: Problem, Solution, Tech & Features:

*"In traditional logistics, tracking item movement across multiple checkpoints often suffers from manual errors and data discrepancies. To solve this, I built a centralized **Supply Chain Product Tracking backend using Spring Boot and PostgreSQL**.*

*The system exposes 5 key REST endpoints: registering new products with automated default statuses, viewing all inventory, updating milestone stages, and removing discontinued items. I used **Spring Data JPA with Hibernate** for database operations and structured the application into clean Controllers, Business Services, DTOs with Jakarta validation, and a Global Exception Handler. This project originally started in Node.js, and I migrated it to Spring Boot to leverage enterprise-level typing, maintainability, and clean 3-tier architecture."*

3. The 2-Minute In-Depth Pitch (When the interviewer is a Senior Java Technical Lead)

2-Minute Pitch: Comprehensive Architecture & Workflow:

*"My project is a **Supply Chain Product Tracking System** engineered with **Spring Boot 3 and PostgreSQL**. The core business challenge is providing real-time lifecycle tracking of physical goods as they move from factory production to end delivery.*

***Architecture:** I followed a strict 3-tier enterprise architecture. When an HTTP request arrives, it hits my **ProductController**, where incoming JSON is validated using DTOs and Jakarta annotations like @NotBlank. The controller delegates execution to **ProductServiceImpl**, which encapsulates the business rules-such as ensuring a new product defaults to 'Manufactured' and guaranteeing that status updates never accidentally overwrite the product's name or origin.*

***Data Layer:** The service communicates with PostgreSQL via **ProductRepository** extending Spring Data's JpaRepository. In PostgreSQL, products are mapped using a JPA Entity with an auto-incrementing identity ID.*

***Migration Experience:** A key technical highlight was migrating the backend from Node.js/Express to Spring Boot while maintaining 100% API contract and database schema compatibility on port 5000."*

Why did you choose this project?	Supply chain visibility is a mission-critical domain in enterprise IT. I wanted hands-on experience building scalable CRUD REST APIs with Spring Data JPA and relational databases.
What was your individual role?	Sole backend developer. Designed the PostgreSQL schema, mapped JPA entities, coded all Controllers, DTOs, and Services, and configured database pooling in application.properties.
Key learning & challenge faced?	Learned how Spring Boot auto-configuration and Hibernate simplify boilerplate JDBC code. A key challenge was ensuring that PUT status updates strictly modified only the status field without altering product origin.

PAGE 2 - Complete Project Architecture and Workflow

Understand the end-to-end internal flow from client request to PostgreSQL database persistence.

1. High-Level 3-Tier Layered Architecture

Layer	Component in Project	Core Responsibility
1. Client / Frontend	Browser / Postman / Client App	Sends JSON HTTP requests to http://localhost:5000 and receives JSON responses.
2. Controller Layer	RootController ProductController	Entry point for HTTP requests. Handles URL mapping, path variables, request body deserialization, and validates DTOs.
3. DTO Layer	ProductRequestDto StatusUpdateDto	Decouples internal DB entities from external API payloads. Uses @NotBlank to validate input fields.
4. Service Layer	ProductService (Interface) ProductServiceImpl (Class)	Encapsulates business logic, enforces defaults ('Manufactured'), and prevents accidental column mutations.
5. Repository Layer	ProductRepository (extends JpaRepository)	Data access abstraction. Generates SQL queries dynamically for CRUD operations without raw SQL code.
6. Entity Layer	Product.java	JPA entity mapping the Java object to the PostgreSQL products table via Hibernate ORM.
7. Database	**PostgreSQL** (Port 5432)	Relational database storing persistent table rows with auto-incrementing id SERIAL PRIMARY KEY.

2. Complete End-to-End Request Flow (Step-by-Step for All 5 Operations)

Operation	Endpoint & Method	Internal Step-by-Step Execution Flow	Response Code
Health Check	GET /	Client pings server -> RootController.healthCheck() intercepts -> Returns plain text string "Server is running!" directly without hitting DB.	200 OK
Register Product	POST /products	1. Client sends JSON {name, origin, status}. 2. ProductController validates @Valid ProductRequestDto. 3. ProductServiceImpl sets status to 'Manufactured' if blank and builds Product entity. 4. ProductRepository.save(product) executes PostgreSQL INSERT. 5. Generated ID is assigned and saved entity returned as JSON.	200 OK (JSON Product with ID)
View All Products	GET /products	1. Client requests inventory. 2. ProductController calls productService.getAllProducts(). 3. ProductRepository.findAll() triggers SELECT * FROM products. 4. Hibernate converts SQL result set into List<Product>. 5. Controller serializes list into a JSON array.	200 OK (JSON Array)
Update Status	PUT /products/{id}/status	1. Client sends target ID in URL and JSON {status: "Shipped"} in body. 2. ProductServiceImpl.updateProductStatus() searches product by ID using findById(id). 3. If not found, throws ResourceNotFoundException (404). 4. If found, sets **ONLY** the new status, preserving original name and origin. 5. Calls save() which executes SQL UPDATE.	200 OK (Updated Product JSON)
Delete Product	DELETE /products/{id}	1. Client sends ID. 2. Service checks existence via findById(id) (throws 404 if missing). 3. Calls productRepository.delete(product) which executes SQL DELETE FROM products WHERE id = ?. 4. Controller returns JSON: {"message": "Product deleted successfully"}.	200 OK (Confirmation JSON)

3. The Product Lifecycle & Logistics State Model

Milestone State Progression: [Manufactured] -> [Shipped] -> [Delivered]
- **Manufactured:** Product has finished assembly at the factory/plant and is registered in the database.
- **Shipped:** Product has been handed over to dispatch and is in transit across distribution channels.
- **Delivered:** Product has reached the destination warehouse or customer doorstep.

Why is milestone status tracking important in Viva? Interviewers love this because it reflects real-world supply chain visibility: preventing inventory loss, identifying transit bottlenecks, and providing accurate delivery estimates to stakeholders.

PAGE 3 - Technology Stack Explained for Viva

Comprehensive guide to each technology, real-world analogies, and core interview comparison questions.

1. The 10 Core Technologies Deep Dive

Technology	What & Why Used in Project	Interview Viva Question & Quick Answer
Java 21	Modern LTS release providing high performance, enhanced pattern matching, record classes, and virtual threads.	**Q: Why use Java for enterprise backends?** **A:** Platform-independence, strong type safety, robust memory management, and rich enterprise ecosystem.
Spring Boot 3.x	Opinionated framework on top of Spring. Uses starter dependencies and auto-configuration to eliminate boilerplate XML.	**Q: What are key benefits of Spring Boot?** **A:** Embedded Tomcat server, auto-configuration, production metrics, and rapid development setup.
Spring MVC	Model-View-Controller module in Spring. Handles incoming HTTP requests via DispatcherServlet and routes them to Controller methods.	**Q: What is the role of DispatcherServlet?** **A:** It is the front controller that intercepts all incoming HTTP requests and routes them to appropriate handlers.
REST API	Architectural style using standard HTTP verbs (GET, POST, PUT, DELETE) and JSON for stateless client-server communication.	**Q: What does 'stateless' mean in REST?** **A:** The server stores no client session context between requests; each request contains all info required to process it.
Spring Data JPA	Data access abstraction layer that eliminates boilerplate DAO code by providing interfaces like JpaRepository with auto-implemented CRUD.	**Q: How does JpaRepository work without implementation?** **A:** Spring dynamically creates proxy implementations at runtime based on method names and JPA conventions.
Hibernate	Leading Object-Relational Mapping (ORM) provider implementing the JPA standard. Automatically translates Java objects into SQL queries.	**Q: What problem does an ORM solve?** **A:** Bridges the object-relational impedance mismatch between object-oriented Java classes and relational SQL tables.
PostgreSQL	Powerful open-source relational database known for ACID compliance, reliability, and robust support for concurrent transactions.	**Q: What ensures data reliability in PostgreSQL?** **A:** Strict ACID compliance (Atomicity, Consistency, Isolation, Durability) and write-ahead logging (WAL).
Apache Maven	Build automation and dependency management tool using pom.xml. Handles library downloads, compilation, packaging, and testing.	**Q: What is the Maven build lifecycle?** **A:** A sequence of phases: validate -> compile -> test -> package -> verify -> install -> deploy.
PostgreSQL JDBC Driver	Database adapter library allowing Java applications to communicate with PostgreSQL over TCP/IP using standard JDBC APIs.	**Q: What happens if JDBC driver is missing?** **A:** Java cannot establish a raw socket connection to the database, throwing ClassNotFoundException.
JSON	JavaScript Object Notation: lightweight, human-readable text format used for data exchange between client and backend REST endpoints.	**Q: How does Spring Boot parse JSON?** **A:** It uses the **Jackson** library by default to serialize Java objects to JSON and deserialize JSON to DTOs.

2. Top 7 Interview Comparison Questions (Frequently Asked in Service Companies)

Comparison	Key Distinctions to Explain in Viva
Spring vs Spring Boot	**Spring** is a complete Java EE enterprise framework requiring extensive XML/Java configuration and manual server deployment. **Spring Boot** is an opinionated extension of Spring that provides starter dependencies, auto-configuration, and an embedded Tomcat server for immediate out-of-the-box execution.
Spring Boot vs Spring MVC	**Spring MVC** is a specific web framework within Spring for building web applications and REST APIs using DispatcherServlet. **Spring Boot** is an umbrella project that packages Spring MVC along with automated setup, configuration, and runtime dependencies.
JPA vs Hibernate	**JPA** (Jakarta Persistence API) is a standard Java *specification* or interface defining ORM rules. **Hibernate** is the actual concrete *implementation* that executes those rules and generates the underlying SQL statements.
JDBC vs JPA	**JDBC** is low-level and requires manual SQL strings, opening/closing connections, and mapping ResultSet rows manually. **JPA** is high-level; it operates directly on Java objects (Entities) and handles SQL generation and object mapping automatically.
SQL vs PostgreSQL	**SQL** is the universal standard query language for relational databases. **PostgreSQL** is a specific Relational Database Management System (RDBMS) software that executes SQL statements and stores data on disk.
Maven vs npm	**Maven** is the build and dependency manager for Java projects using pom.xml. **npm** is the package manager for JavaScript/Node.js projects using package.json. When migrating this project, Maven replaced npm.
REST API vs Traditional Web App	**Traditional web apps** render HTML pages on the server (JSP, Thymeleaf) and send full markup to the browser. A **REST API** sends raw data (JSON), allowing any client (React, Vue, mobile apps, IoT sensors) to consume it independently.

PAGE 4 - Project Implementation & Important Code Concepts

Codebase walkthrough, core Spring Boot annotations, and architectural separation of concerns.

1. Codebase Structure & File Responsibilities

File / Class Name	Package / Path	Actual Code Responsibility in this Project
ProductTrackingApplication.java	com.producttracking	Main entry point containing main() method with SpringApplication.run().
Product.java	com.producttracking.entity	JPA Entity mapped to table products with fields id, name, origin, and status.
ProductRepository.java	com.producttracking.repository	Interface extending JpaRepository<Product, Long> for built-in database CRUD operations.
ProductService.java	com.producttracking.service	Business interface defining contract methods (getAllProducts, createProduct, etc.).
ProductServiceImpl.java	com.producttracking.service.impl	Implements business logic: enforces defaults and ensures status updates protect name/origin.
ProductController.java	com.producttracking.controller	REST Controller handling /products routing, DTO validation, and returning ResponseEntity.
RootController.java	com.producttracking.controller	Handles GET / health check returning plain text "Server is running!".
ProductRequestDto.java	com.producttracking.dto	DTO carrying input data for product creation with @NotBlank validations on name and origin.
StatusUpdateDto.java	com.producttracking.dto	DTO carrying payload for status changes with @NotBlank validation on status.
CorsConfig.java	com.producttracking.config	Implements WebMvcConfigurer allowing frontend origins to access APIs on port 5000.
application.properties	src/main/resources	Configures port 5000, PostgreSQL datasource URL, DB credentials, and Hibernate DDL update.

2. Spring Boot Annotations Demystified for Interviews

Annotation	Meaning & Why Used	Where It Appears in Project
@SpringBootApplication	Enables auto-configuration, component scanning, and property support. Combines @Configuration, @EnableAutoConfiguration, and @ComponentScan.	ProductTrackingApplication.java
@RestController	Marks class as a REST endpoint handler. Combines @Controller and @ResponseBody, ensuring returned objects serialize directly to JSON.	ProductController, RootController
@RequestMapping	Sets base URL path mapping for all handler methods within the controller class.	@RequestMapping("/products")
@GetMapping, @PostMapping @PutMapping, @DeleteMapping	Composed shortcut annotations mapping specific HTTP GET, POST, PUT, and DELETE methods to specific controller handler functions.	All endpoints inside ProductController
@Service / @Repository	Specialized @Component stereotypes indicating business service layer and database DAO access layer respectively. Enables Spring IoC component scanning.	ProductServiceImpl.java ProductRepository.java
@Entity / @Table	Specifies that the Java class is a persistent JPA entity mapped to the PostgreSQL table named products.	Product.java
@Id / @GeneratedValue	Marks primary key field. GenerationType.IDENTITY leverages PostgreSQL's native auto-incrementing SERIAL sequence.	id field in Product.java
@RequestBody / @PathVariable	@RequestBody binds incoming HTTP JSON body to DTO. @PathVariable extracts parameters directly from URL path (e.g. {id}).	Parameters in ProductController

3. Controller vs Service vs Repository in this Project

Interview Golden Rule: Separation of Concerns

- **Controller (ProductController):** Handles HTTP layer only (URL matching, JSON parsing, HTTP status codes 200/400/404). *Never put business logic or DB calls here!*
- **Service (ProductServiceImpl):** Handles business logic (e.g., verifying product exists, setting default status to 'Manufactured', ensuring status update does not overwrite name/origin).
- **Repository (ProductRepository):** Handles data persistence only (translating method calls into PostgreSQL SQL queries via Spring Data JPA).

PAGE 5 - Technical Viva Questions from the Project

Top 20 high-frequency viva interview questions with crisp, verbal-ready answers.

Part A: Project-Based Questions

1. What is your project about?
Centralized RESTful backend developed with Spring Boot and PostgreSQL to track product supply chain lifecycles from manufacturing to delivery.

2. What real-world problem does it solve?
Eliminates manual logging discrepancies and delayed tracking in warehouses by providing real-time, milestone status updates for logistics.

3. Why did you select PostgreSQL instead of MySQL?
PostgreSQL offers superior adherence to strict SQL standards, advanced concurrency control (MVCC), and high reliability for transactional tracking.

4. What are the main modules in your application?
Controller module (HTTP routing), Service module (business rules), Repository module (data access), DTO module (input validation), and Config module (CORS).

5. How does product tracking actually work?
When created, status defaults to 'Manufactured'. As items move through logistics, warehouse operators hit PUT endpoint with product ID to advance status to 'Shipped' or 'Delivered'.

6. How is product data stored in the database?
Inside PostgreSQL table products with columns: id (BIGINT PK), name (VARCHAR), origin (VARCHAR), and status (VARCHAR).

7. How does the application connect to PostgreSQL?
Via Spring Boot's HikariCP connection pool configured in application.properties using the PostgreSQL JDBC driver and standard DB credentials.

8. What happens when a product ID is not found?
findById(id) returns Optional.empty(), throwing custom ResourceNotFoundException, returning a clean 404 response.

9. How do you protect product name during status update?
In ProductServiceImpl, status update method fetches existing entity and executes existingProduct.setStatus(dto.getStatus()), leaving name/origin untouched.

10. How are product IDs generated?
Via @GeneratedValue(strategy = IDENTITY), which delegates auto-incrementing integer key generation to PostgreSQL's native SERIAL sequence.

Part B: Core Technical Questions

11. What is Dependency Injection (DI)?
A design pattern where an object receives its dependencies from an external container (Spring IoC) rather than instantiating them directly with new.

12. What is Inversion of Control (IoC)?
The architectural principle where object creation, lifecycle, and wiring is inverted from the developer to the Spring framework container.

13. Why use Constructor Injection over @Autowired on fields?
Constructor injection ensures dependencies are immutable (final), makes classes easily testable, and prevents uninitialized object instances.

14. What is an ORM?
Object-Relational Mapping (ORM) maps Java classes to database tables and object instances to table rows, removing manual SQL query coding.

15. What is the difference between JPA and Hibernate?
JPA is the standard Java specification/interface; Hibernate is the actual ORM library that implements JPA and converts operations into SQL queries.

16. What is a DTO and why not expose the Entity directly?
DTOs (Data Transfer Objects) encapsulate incoming API data, preventing over-posting attacks, decoupling DB schema from API contracts, and adding validation.

17. What is the purpose of application.properties?
Spring Boot's central configuration file used to specify server ports, database URLs, credentials, JPA behaviors, and logging levels without changing code.

18. What is the difference between HTTP POST and PUT?
POST creates a new resource on the server (non-idempotent). PUT updates or replaces an existing resource identified by a specific URI (idempotent).

19. What is the difference between HTTP GET and DELETE?
GET retrieves data without modifying server state (safe/idempotent). DELETE removes the specified resource from the server (idempotent).

20. What is CORS and why is CorsConfig.java necessary?
Cross-Origin Resource Sharing is a browser security mechanism. CorsConfig allows frontend apps on other domains/ports to access port 5000 safely.

PAGE 6 - Cross-Questioning & Scenario-Based Viva

Detailed answers for the tricky follow-up questions interviewers ask to test your practical depth.

1. Why did you migrate from Node.js/Express to Spring Boot?

Answer: Node.js is lightweight, but Spring Boot provides enterprise-grade type safety, structured 3-tier architecture, built-in dependency injection, and Spring Data JPA. For enterprise supply chains requiring transactional integrity and team scalability, Spring Boot is far more robust.

Follow-up: Did you have to change your frontend? No, because we preserved the exact same HTTP routes, request bodies, JSON response formats, and port 5000.

2. Why separate Controller, Service, and Repository instead of writing DB queries in Controller?

Answer: Separation of Concerns. Controllers should only handle web/HTTP concerns. If business rules change (e.g. adding a new tracking status), we only modify the Service layer without touching the HTTP layer, ensuring maintainability and clean unit testing.

Follow-up: What principle is this? The Single Responsibility Principle (SRP from SOLID).

3. What happens behind the scenes when a user sends POST /products?

Answer: 1. Request hits Tomcat on port 5000 -> 2. DispatcherServlet routes to ProductController.createProduct() -> 3. Jackson deserializes JSON into ProductRequestDto and validates @NotBlank -> 4. Service applies business defaults -> 5. Repository calls Hibernate -> 6. Hibernate executes SQL INSERT via JDBC into PostgreSQL -> 7. Generated ID is assigned and returned as JSON.

4. What happens if the PostgreSQL database goes down during application runtime?

Answer: HikariCP connection pool attempts to reconnect. If unavailable, it throws a CannotCreateTransactionException or JDBCConnectionException. Our GlobalExceptionHandler catches it and returns a clean 500 Internal Server Error without exposing DB passwords or stack traces.

Follow-up: How to monitor DB health? By adding Spring Boot Actuator with /actuator/health.

5. What happens if someone submits an invalid ID format like PUT /products/abc/status?

Answer: Spring's DispatcherServlet fails to convert string 'abc' to Long. It triggers a MethodArgumentTypeMismatchException. Our centralized exception handler intercepts it and returns HTTP 400 Bad Request with a clear message: 'Invalid parameter: id must be of type Long'.

6. How would you prevent invalid backward status updates (e.g. Delivered -> Manufactured)?

Answer: In ProductServiceImpl, implement a state transition validator: if current status is 'Delivered', reject updates to 'Manufactured' or 'Shipped' by throwing an IllegalStateException (HTTP 400).

7. How would you track the complete audit history (timestamp, location, user) of each status change?

Answer: Create a secondary entity named ProductHistory with fields: id, productId, previousStatus, newStatus, timestamp, and updatedBy. Every time updateProductStatus() is called, insert a new record into product_history within the same @Transactional method.

8. How would you add user authentication to this system?

Answer: Add spring-boot-starter-security with JWT (JSON Web Tokens). Create a User entity with roles (e.g. ROLE_WAREHOUSE, ROLE_ADMIN). Secure endpoints so only authenticated warehouse personnel can call POST, PUT, and DELETE.

9. How would you support hundreds of thousands of products without slowing down GET /products?

Answer: Implement **Pagination and Sorting** using Spring Data's Pageable: Page<Product> findAll(Pageable pageable). The client can request GET /products?page=0&size;=20, preventing out-of-memory errors by fetching small batches.

10. How would you deploy this application in a production environment?

Answer: Build the standalone executable JAR using mvn clean package -DskipTests, package it into a lightweight Docker container running OpenJDK 21, and deploy it to a cloud VM (AWS EC2 / Azure) pointing to an AWS RDS PostgreSQL instance.

PAGE 7 - Final Interview Revision Sheet & Cheat Sheet

Fast-revision summary sheet to review 30 minutes before your placement interview.

1. Rapid Project Fact Sheet

Project Name: Supply Chain Product Tracking System	**Target Port:** 5000 (http://localhost:5000)
Language: Java 21 (LTS)	**Build Tool:** Apache Maven (pom.xml)
Framework: Spring Boot 3.3.5 (Spring MVC)	**Database:** PostgreSQL 16 (Port 5432)
ORM Engine: Hibernate 6 (via Spring Data JPA)	**Connection Pool:** HikariCP (built-in default)
Main DB Table: products (id, name, origin, status)	**Key Lifecycle:** Manufactured -> Shipped -> Delivered
Primary Endpoints: GET /, GET /products, POST /products, PUT /products/{id}/status, DELETE /products/{id}	**Architecture:** 3-Tier Layered Architecture (Controller-Service-Repo)

2. Technical Keywords in Plain English (1-Line Definitions)

REST API	Stateless web architectural style exchanging raw data (JSON) via standard HTTP verbs (GET, POST, PUT, DELETE).
Dependency Injection (DI)	Passing required dependencies into a class rather than the class creating them directly with 'new'.
Inversion of Control (IoC)	The framework manages object lifecycles and wiring instead of manual developer control.
JPA (Jakarta Persistence API)	Standard Java specification interface for object-relational mapping.
Hibernate	Popular ORM library implementing JPA specifications to translate Java objects into SQL queries.
Entity	A Java class annotated with @Entity representing a relational database table row.
Repository	A Spring Data interface (JpaRepository) providing out-of-the-box database CRUD operations.
DTO (Data Transfer Object)	An object dedicated to carrying data across network boundaries without exposing database entities.
Controller	Web-layer class annotated with @RestController that maps incoming HTTP requests to methods.
Service	Class holding core business logic, validation rules, and transaction boundaries (@Service).
JDBC Driver	Database-specific communication adapter translating Java database calls into network socket protocols.
Maven	Project management tool handling dependencies, compilation, and building executable JAR packages.

3. Senior Placement Trainer's Interview Advice

Trainer Pro-Tips for Service-Company Interviews

- **How to start confidently:** Speak clearly. Begin with your 1-minute pitch. Frame the project around solving a logistics tracking problem.
- **How to answer when you don't know:** Never guess blindly or say 'I don't know'. Instead say: *"I haven't implemented that specific feature yet, but based on my understanding of Spring Boot, it would involve..."*
- **Own your individual contribution:** State proudly that you wrote the Java controllers, services, JPA mappings, and tested all endpoints on port 5000.
- **5 Mistakes to Avoid:** (1) Memorizing code lines instead of understanding request flow; (2) Confusing JPA with Hibernate; (3) Saying business logic is inside the Controller; (4) Claiming you used advanced microservices when this is a clean, solid monolith; (5) Talking too fast.

Pre-Interview Confidence Checklist:

[✔] Can I explain the 1-minute project pitch fluently without hesitation?
[✔] Can I trace a request step-by-step from POST /products down to the PostgreSQL INSERT?
[✔] Do I know the difference between @RestController, @Service, and @Repository?
[✔] Can I explain why we migrated from Node.js to Spring Boot for enterprise type safety and structure?